Surviving Quantum Mutants from QuCheck and Muskit Archives: Equivalent Builds, Input Limitations and Lost Histogram Information

Seojin Choi[1] and Geunseok Yang[1*]

[1]Department of Computer Science and Engineering, Hankyong National University, Anseong, Republic of Korea.

*Corresponding author. E-mail: gsyang@hknu.ac.kr
Contributing author: insui12@hknu.ac.kr

Abstract

Undetected quantum mutants can preserve behaviour, differ only on forbidden inputs or evade the statistics used by a test, so detection counts alone do not identify which test changes could help. We distinguish these causes in supplied archived builds, analysing 120 subjects from QuCheck, a property-based testing framework, and 1,136 from Muskit, a mutation-testing tool. An executor contract accepts every computational-basis input; a slot-consistent contract keeps selected workspace qubits initially zero without excluding supplied preparations. Muskit's admissible-input witnesses, whose output states change under an admissible input, fall from 130 to 8 under the slot-consistent contract, without losing any detection by the original tests. Under documented input restrictions, QuCheck's 50 subjects undetected by the test catalogue comprise 41 equivalent-build subjects and 9 that differ only outside those restrictions. Full-histogram comparison detects all 50 Muskit subjects whose output distributions differ while single-qubit marginals and parity agree. A search over admissible inputs that adds pairwise parities and then full histograms, with new shots at the original per-test allocation, detects 118 of 130 Muskit executor admissible-input witnesses. The original total shot budget spread across that search cannot detect any of them under the executor contract. Full-shot search detects all 8 slot-consistent admissible-input witnesses with existing statistics, while fixed-budget search does so after adding pairwise parities. These exploratory results concern archived small noiseless builds and stated contracts, so testers should establish admissible inputs and inspect full histograms before interpreting non-detection as equivalence.

Keywords: quantum software testing, mutation testing, input contracts, test oracles, circuit equivalence

1 Introduction

Mutation testing evaluates tests using deliberately changed programs (Jia & Harman, 2011). We examine undetected quantum mutants because an edit may preserve behaviour or evade what a test records; either case changes how survival informs test adequacy in quantum software quality assurance. Equivalent mutants complicate the interpretation of detection counts, and quantum mutation studies relate survival to oracle choice, edit type, location and program traits (Fortunato et al., 2022a; Mendiluze Usandizaga et al., 2025). An input contract specifies admissible inputs and initial register states, yet a detection count alone does not say whether a tester should change the input, change the oracle's statistics or expect the same circuit behaviour.

We reuse archived mutants from QuCheck, a property-based testing framework (Pontolillo et al., 2026), and Muskit, a mutation-testing tool (Mendiluze et al., 2021), treating each provided mutant as a subject. Muskit's archive contains three programs: conditional execution (CE), Little Bernstein–Vazirani (LittleBV) and quantum random-access memory (QRAM). For example, a LittleBV subject inserts a controlled-Z gate between qubits indexed 2 and 5 using indices starting at 0, but the default input leaves every qubit zero and hides the edit. Setting only qubit 2 to 1 instead makes the measured probability on qubit 5 differ between the subject and its unmutated program, called its release control. The numerical example checks this difference against the detection threshold and the admissible input domain in Section 3.3.

Our research questions (RQs) distinguish survival causes from test changes: RQ1 separates equivalent builds, input restrictions and information discarded by statistics or measurement bases, while RQ2 compares input changes, richer statistics and wider measurement bases together with shot costs. Silent subjects are mutants undetected by every test in the catalogue. Table 1 defines the census classes, and sampled outcomes enter those classes through the workflow in Fig. 1. This study makes two contributions:

- We classify survival causes, separating equivalent builds, input-contract limits and lost histogram information, and distinguishing full-circuit equivalence from agreement on the default all-zero input in Section 4.
- We compare input contracts and test changes, reporting exact-probability differences separately from detections using new shots and their costs in Section 5.

Circuit-equivalence checking (Burgholzer & Wille, 2021; Campos & Miranskyy, 2026) and suite selection (Wang et al., 2022) address parts of this problem, while our census links each cause of non-detection to a corresponding test change and its measured shot cost within the supplied builds and input contracts. Section 2 reviews the background and Section 3 specifies the study design, before Sections 4 and 5 answer the RQs. Discussion, threats to validity and conclusions follow in Sections 6–8.

2 Background and related work

We position the census against quantum testing, mutation analysis and circuit-equivalence methods. A quantum test does not read out an entire state in one shot, because each shot yields a bit string and repeated shots give a histogram of how

Table 1 The census separates survival causes by behaviour and test information

Class	Name	What differs from the release control?
E	Equivalent build	Full circuits agree up to a physically irrelevant global phase.
A	Admissible-input witness	Some permitted basis input changes the output state.
C	Contract-limited	Outputs agree on all permitted inputs up to one global phase; only forbidden inputs differ.
P	Phase-only	Each permitted basis output agrees up to phase; relative phases between inputs differ.
D	Detected	An exact difference in a statistic used by the tests exceeds its detection threshold.
O	Summary-limited	A full output distribution differs, but the statistics used by the tests hide it.
M	Measurement-basis-limited	Output states differ, but the measurement bases used by the tests hide it.
T	Threshold-limited	A statistic gap is positive but no greater than its detection threshold.
I	Input-limited total	Aggregate of admissible-input witness, contract-limited and phase-only classes.

I aggregates A, C and P and is not counted twice. Formal numerical criteria follow in Section 3.3.

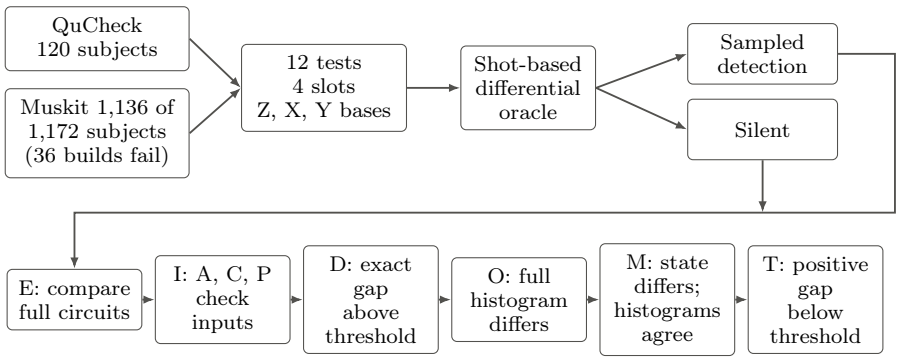


Fig. 1 Shot-based differential tests in the Z, X and Y measurement bases lead to sampled detections or silent subjects, both assigned to the first matching exact-probability class (Table 1), with arrows between classes meaning "otherwise"; a slot fixes circuit arguments or an input preparation (Section 3.2)

often each string occurs. An oracle can then keep a few statistics of that histogram or compare the full histogram. Quantum test systems already address inputs, output probabilities and suite cost, whereas we ask why mutants survive in their fixed test catalogues. QSharpCheck generates constrained inputs and checks Q# properties (Honarvar et al., 2020). QuCheck adds Qiskit generators, assertions and preconditions

(Pontolillo et al., 2025, 2026), with Z, X and Y tests using single-qubit marginals, statistical decisions, multiplicity correction and duplicate removal (Pontolillo et al., 2025, Sections 4.4.1–4.4.3). Statistical assertions check expected state patterns (Huang & Martonosi, 2019), while MorphQ tests Qiskit through metamorphic links between program outputs (Paltenghi & Pradel, 2023); a survey reviews these approaches (Paltenghi & Pradel, 2024).

Quito uses input, output and joint coverage with wrong-output and specified-probability oracles, tracking faults found and execution cost, but its value-based criteria exclude phase testing (Ali et al., 2021). MutTG searches for suites that trade off size against mutants detected, using the current correct program for expected outputs and matching search counts for random search (Wang et al., 2022). Fortunato et al. (2022a) also discuss equivalent mutants and oracle choice in a Qiskit case study. Quito and MutTG compare suites by detection counts, whereas the census separates survival causes under explicit input contracts. We connect equivalence calculations to admissible-input and statistic limitations, then measure the shot cost of detecting those differences.

Classical work studies trade-offs between tests and resources (Yoo & Harman, 2012) and size and overlap threats in mutation testing (Papadakis et al., 2019); quantum-testing guidance also calls for fair baselines and clear settings (Li et al., 2026). Muskit and QMutPy provide quantum mutation systems (Fortunato et al., 2022b; Mendiluze et al., 2021), and all subjects here come from the pinned QuCheck and Muskit archives, with none generated for this study or selected by detection outcome. Mutant survival varies with edit type, location and program traits (Mendiluze Usandizaga et al., 2025), so we report both subject counts and behaviour-group counts, with the source-edit classes reported for these builds in Online Resource 1, Table S20.

Circuit-equivalence checkers address semantic questions beyond finite test outcomes (Burgholzer & Wille, 2021), and transpiler-based equivalence checking (Campos & Miranskyy, 2026) conservatively identifies full-circuit equivalence when matched transpilation produces identical Open Quantum Assembly Language (OpenQASM) circuit text, assuming a semantics-preserving transpiler. Its experimental state-vector reference compares one shared input state; our full-circuit calculation checks the supplied wrapper builds, and our state calculation checks outputs from the catalogue input, the all-zero state before the build runs. These standard calculations support the joint census and contract comparison without supplying an independent program specification. Saad et al. (2026) compare mutants with real faults, showing how well mutants represent faults rather than whether tests detect them. The real-fault detection performance of our differential oracle remains unmeasured. Online Resource 1, Table S4 gives the full comparison and identifies the inspected related work.

3 Study design

This exploratory, post hoc re-analysis of two archived mutant sets classifies survival and measures the cost of detecting differences.

3.1 Research questions

The RQs separate the cause of a non-detection from the effectiveness and cost of a test change.

- **RQ1: Why do subjects survive the test catalogue?** We measure subject and behaviour-group counts under each input contract, distinguishing full-circuit equivalence, input limits and lost measurement information.
- **RQ2: Which changes to tests detect silent subjects?** We measure exact-law detection counts, fresh-shot detection frequencies, set-wide false-positive frequencies and shot allocations for changes to inputs, statistics and measurement bases.

3.2 Subjects, definitions and input contracts

The following definitions specify the analysis units, test settings and admissible inputs. A subject is one provided mutant. Its release control is the given unmutated program. The test catalogue is the complete set of fixed differential tests applied to each subject. A slot fixes circuit arguments or an input preparation, such as flipping the lowest qubit before the program runs, and each test pairs one slot with one output measurement basis, Z, X or Y. A silent subject survives every test in that test catalogue, and a census classifies every subject by the cause of its detection or survival. A mutant set contains the subjects checked together. Our fixed wrapper adds fixed preparations and common differential tests to archived circuits, and each resulting full circuit is a build. A slot preparation is the fixed state preparation specified by a slot. A behaviour group contains subjects from the same program whose builds have full-circuit equivalence up to global phase in every slot within the numerical tolerance. The shot-based differential oracle compares single-qubit marginals and whole-register odd parity, the existing statistics, whereas a full histogram retains all measured output bit strings.

The catalogue input is the all-zero state before a build, while the application input is the register state after slot preparation, before the archived program operations (the program body). The executor, or circuit runner, admits all computational-basis bit strings under the executor contract. A workspace contract assumes that selected auxiliary qubits, called workspace qubits, initially hold $|0\rangle$, and its mask lists the qubits subject to this assumption. A mask is slot-consistent when it fixes no qubit varied by a slot or by an input preparation documented in source or properties before the program body. The program-semantic contract admits only inputs satisfying restrictions documented in source, properties or specifications. The admissible span contains all linear combinations of admissible basis states, while a coherent input uses a superposition of such states with definite phases relative to each other.

With indices starting at 0, we write q_r for qubit r, $q_{r:s}$ for the inclusive range from q_r through q_s, and $z_r \in \{0, 1\}$ for the measured bit of q_r in the selected basis. The slot-consistency audit enumerates admissible slots and reclassifies every subject

(Online Resource 1, Table S46). The audited upper-register mask, starting at q_3, fixes the fewest workspace qubits preserving every preparation: $q_{3:5}$ for CE and LittleBV and $q_{3:6}$ for QRAM, each initially in $|0\rangle$, leaving $q_{0:2}$ as a common input interface. Fixing LittleBV q_2 preserves its slots but adds an undocumented fourth fixed qubit; these assumed masks do not recover a unique semantic specification. An admissible input obeys the chosen contract, whereas a contract-violating state does not satisfy its restrictions on register states.

QuCheck's 120 reported subjects come from Deutsch–Jozsa, Grover, teleportation and superdense coding; the last pair share Bell preparation and form a 60-subject lineage, a group of programs sharing source code. A further 60 development subjects from Fourier transform and phase estimation were used only to select the earlier test suites (Online Resource 1, Table S5). Muskit's archive contains 1,172 subjects, of which 36 fail to build, leaving 1,136 buildable subjects after excluding 10 invalid-qubit-index and 26 invalid-gate-arity cases. Both RQs use the test catalogue for the included subjects. Table 2 states the slots and admissible domains; per-program subject and group counts are in Online Resource 1, Table S19 and Tables S47–S49.

H, S and X denote Hadamard, $\mathrm{diag}(1, i)$ and bit flip, with $i^2 = -1$; Z, X and Y also label the Pauli measurement axes, so X denotes the same Pauli operator in either use. The symbol † denotes conjugate transpose. Each test has index $t = 3\ell + b$, where ℓ indexes the slot and $b = 0, 1, 2$ denotes Z, X and Y: Z adds no output rotation, X adds H, and Y adds $S^\dagger$ then H. Muskit's slots apply X to each set bit of the integers 0, 1, 2, 3 before the program body. The symbol $\oplus$ denotes exclusive OR. For application input $|\xi\rangle$ after slot ℓ, the input before the build is $|\xi \oplus \ell\rangle$, where integer kets denote complete registers in little-endian order, so the lowest-index qubit carries the least significant bit. Only q_0 and q_1 vary in Muskit's slot preparations, whereas input search under the executor contract varies every admissible basis input. We use the coherent states

$$|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle), \qquad |+i\rangle = \frac{1}{\sqrt{2}}(|0\rangle + i|1\rangle).$$

3.3 Shot decisions and numerical census

The census separates observed shot decisions from numerical comparisons of the supplied builds, using $N = 8{,}192$ shots per test for each subject and release control. For subject m and test t, $\hat{p}_j^{(m)}$ and $\hat{p}_j^{(c)}$ estimate statistic j in their measured histograms; the test index t is suppressed on both estimated and exact probabilities. For a number of comparisons K and significance level α, the threshold and sampled detection indicator are

$$\begin{aligned} \mathcal{D}(m,t) &= \mathbf{1}[\hat{d}_{m,t} > \tau(K,N)], \quad \hat{d}_{m,t} = \max_j |\hat{p}_j^{(m)} - \hat{p}_j^{(c)}|, \\ \tau(K,N) &= \sqrt{2\ln(4K/\alpha)/N}. \end{aligned} \tag{1}$$

The indicator $\mathbf{1}$ is one when its predicate holds and zero otherwise, and ties at a detection threshold count as non-detections. The test catalogue uses $K = 120$, $\alpha = 0.001$ and $\tau_0 = 0.056513$ in Eq. (1), where K rounds up $(12 + 1)(8 + 1) = 117$

Table 2 QuCheck and Muskit builds differ in their slot preparations and admissible input domains

Program (qubits)	Slot parameters	Admissible domain; fixed initialisation
QuCheck		
Deutsch–Jozsa (3)	$F_{\mathrm{DJ}} = 0, 1, x_0, x_0 \oplus x_1$ in slot order	No quantum input; prepares each of $q_{0:2}$ in $\|0\rangle$; 4 of 8 oracles; oracle $\|x, y\rangle \mapsto \|x, y \oplus F_{\mathrm{DJ}}(x)\rangle$
Grover (3)	Marked integer 0, 1, 2, 3 in $q_1 q_0$ order; 1 iteration	No quantum input; prepares each of $q_{0:2}$ in $\|0\rangle$, with q_2 the workspace qubit
Teleportation (3)	Prepares q_0 in $\|0\rangle, \|1\rangle, \|+\rangle, \|+i\rangle$	Arbitrary data state on q_0; prepares q_1 and q_2 in $\|0\rangle$; quantum-controlled corrections
Superdense coding (2)	Classical messages $\gamma_1\gamma_2 = 00, 01, 10, 11$	These 4 messages; prepares q_0 and q_1 in $\|0\rangle$ before encoding
Muskit		
CE (6)	Common integer slot preparation; internal X, H and phase gates; preparation and uncomputation stay inside the body	Executor contract: all basis strings on $q_{0:5}$; no documented workspace restriction
LittleBV (6)	Common slot preparation varies q_0, q_1; leaves q_2 in $\|0\rangle$	Executor contract: all basis strings on $q_{0:5}$; no documented workspace restriction
QRAM (7)	Common slot preparation; H–phase–H gates on address qubits and memory-bank routing stay inside the body	Executor contract: all basis strings on $q_{0:6}$; no documented workspace restriction

QuCheck contracts follow source properties and builders; Muskit uses the executor contract because no program-semantic restriction is documented. The slot-consistent contract requires the workspace qubits $q_{3:5}$ in CE and LittleBV and $q_{3:6}$ in QRAM to be initially in $|0\rangle$ without excluding slots; both contracts appear in Table 3. For Deutsch–Jozsa, $x = (x_0, x_1)$ contains the input bits, y is the oracle target bit and F_{DJ} is the Boolean oracle. For superdense coding, γ_1, γ_2 are classical message bits. Source evidence and Muskit input preparation: Online Resource 1, Tables S13 and S24.

comparisons: catalogue tests plus a replay, each covering up to eight marginals and whole-register parity. The per-subject Bernoulli union bound is $4Ke^{-N\tau^2/2}$, with the derivation of the comparison count given in Online Resource 1, Section S7.2. QuCheck reuses a release-control histogram among subjects run together for the same program and test, while Muskit reuses one per program and test; records with the same seed are replays rather than independent control draws.

For slot ℓ, $U_{m,\ell}$ and $U_{c,\ell}$ are subject and release-control unitaries on n qubits, whose unitary difference is $u_\ell = |1 - |\operatorname{tr}(U_{m,\ell}^\dagger U_{c,\ell})|/2^n|$, where the trace sums the diagonal entries of the unitary product. Catalogue output states $|\psi_{m,\ell}\rangle, |\psi_{c,\ell}\rangle$ have infidelity $f_\ell = 1 - |\langle\psi_{m,\ell}|\psi_{c,\ell}\rangle|^2$. For exact probabilities, $d_{m,t} = \max_j |p_j^{(m)} - p_j^{(c)}|$, with $t = 3\ell + b$ as defined above. Hats on d denote shot estimates throughout this article and Online

Resource 1. We use $u = \max_\ell u_\ell$ over all slots, and $f = \max_\ell f_\ell$ and $d = \max_t d_{m,t}$ over admissible catalogue slots; the subject index is suppressed on u, f and d. The unitary, state and observable tolerances are $\epsilon_u = 10^{-9}$, $\epsilon_s = 10^{-10}$ and $\epsilon_o = 10^{-12}$, applied to u, f and d, respectively, with the last separating a probability difference from numerical noise. An exact-law detection exceeds a specified statistic threshold using the exact outcome distribution, whereas a state difference defines an admissible-input witness whose statistic gap need not exceed the detection threshold, so these counts can differ. For RQ1, QuCheck's executor-contract census calculates state vectors for additional inputs without shot sampling; RQ2 samples new shots for its search and specificity checks. Behaviour groups are fixed before contract changes and do not count independent defect mechanisms. Algorithm 1 gives the classification procedure; Online Resource 2 supplies the preserved builds, counts, seeds and numerical scripts.

Algorithm 1 Census over the supplied builds

1: Construct $U_{m,\ell}, U_{c,\ell}$; compute unitary difference u over all slots and catalogue f, d on admissible slots; stop for non-unitary builds.
2: If $(u \le \epsilon_u \wedge f > \epsilon_s) \vee (f \le \epsilon_s \wedge d > \epsilon_o)$, stop: conflicting diagnostics.
3: If $u \le \epsilon_u$, assign E.
4: Else if $f \le \epsilon_s$, assign I: A if an admissible basis output changes beyond a global phase; C if the entire admissible span agrees up to one global phase; P if only phases between basis outputs differ.
5: Else if $d > \tau_0$, assign D; record whether sampled $\mathcal{D}(m,t)$ agrees (Section 4.1).
6: Else if $d \le \epsilon_o$ and any catalogue-test full histogram differs, assign O.
7: Else if $d \le \epsilon_o$, assign M and audit other local bases.
8: Else assign T.

Admissible-input search exhausts all admissible slots and basis inputs without stopping at the first distinguishing input. Only illustrative contract-violating states use a scan that stops at the first difference, testing basis states before pairwise superpositions. Teleportation checks the full admissible data subspace (Online Resource 1, Table S24).

To distinguish input limits from lost histogram information, consider the Introduction's LittleBV subject, which has $u = 0.500000 > \epsilon_u$ but catalogue $f = 0.000000$ and $d = 2.22 \times 10^{-16}$ within tolerance, so Algorithm 1 assigns it to the input-limited class. Its distinguishing input $|4\rangle$ sets only the unrestricted q_2. At this input, the Z probabilities of $z_5 = 1$ are 0 for the subject and 1 for its release control, with absolute difference 1.000000, making the subject an admissible-input witness under both Muskit contracts.

A CE subject instead has $u = 0.292893$, $f = 0.500000$ and $d = 2.22 \times 10^{-16}$, so its catalogue state and X histogram differ at slot 0 while existing statistics agree. It is therefore summary-limited under both Muskit contracts, but the Pearson histogram test detects it as described in Section 3.4. The pairwise parity $\Pr(z_2 \oplus z_3 = 1)$ differs by 0.500000 in exact probabilities and 0.498413 in sampled shots (Section 3.4). Source identifiers and seeded examples appear in Online Resource 1, Section S4.2 and Table S22.

3.4 Histogram comparison and fresh-shot search

We compare full histograms and then search admissible inputs under two shot-allocation rules; a role is either the subject or its release control. The split-alpha Pearson comparison gives half of α to existing statistics and half across 12 two-sample Pearson histogram tests, without adding pairs. The existing-statistic threshold rises from τ_0 to $\tau_{\text{split}}(K, N)$, defined by

$$\tau_{\text{split}}(K, N) = \sqrt{2\ln(4K/(\alpha/2))/N}.$$

Here K counts scalar comparisons and N is the number of shots per test per role; catalogue comparisons retain $K = 120$. For equal-size subject and release-control counts a_z, c_z in fixed output-bit-string bins z, the Pearson statistic, degrees of freedom and p-value are

$$\begin{aligned} Q &= \sum_{z:a_z+c_z>0} \frac{(a_z - c_z)^2}{a_z + c_z}, \\ \nu &= |\{z : a_z + c_z > 0\}| - 1, \qquad p_{\text{val}} = \Pr(\chi^2_\nu \geq Q). \end{aligned} \tag{2}$$

We reject if $p_{\text{val}} < \alpha/(2 \times 12)$; with no positive degrees of freedom, $p_{\text{val}} = 1$, and pooled-zero bins are removed without selecting bins by signal or pooling sparse bins.

A law is an exact outcome distribution, written $\boldsymbol{\pi}_m$ for the subject and $\boldsymbol{\pi}_c$ for its release control; multiplying these laws by N gives exact pseudo-counts $N\boldsymbol{\pi}_m, N\boldsymbol{\pi}_c$, used as deterministic diagnostics rather than shot-based tests or estimates of power. The fresh-shot search checks every admissible input and slot in each of the Z, X and Y bases. Each statistic uses fresh draws that are independent for both roles. A rung is a cumulative test family: the input rung uses existing statistics over admissible inputs, and the + pairs rung adds every pairwise parity observable, $\Pr(z_r \oplus z_s = 1)$ for $r < s$. The + histogram rung also adds Pearson histogram tests, without spending α anew at a rung. An elementary test is one statistic at one input, slot and basis: scalar tests include marginals, whole-register parity and pairwise parities, whereas a histogram test compares all output bins. For n total circuit qubits, n_{free} unfixed qubits and N_{hist} searched histograms, $L = N_{\text{hist}}[n+1+\binom{n}{2}]$ counts scalar tests and $J = L+N_{\text{hist}}$ counts all elementary tests. Here $N_{\text{hist}} = 12 \times 2^{n_{\text{free}}}$, covering 4 slots and 3 bases. Executor CE and LittleBV use $n_{\text{free}} = 6$, giving $N_{\text{hist}} = 768$; QRAM uses $n_{\text{free}} = 7$, giving $N_{\text{hist}} = 1{,}536$. All slot-consistent Muskit programs use $n_{\text{free}} = 3$, the three unfixed qubits $q_{0:2}$, giving $N_{\text{hist}} = 96$, whereas QuCheck's specificity searches use $n_{\text{free}} = n$, with n from Table 2, in the same formulas for N_{hist}, L and J. Half of $\alpha = 0.001$ covers the L scalar tests and half covers all N_{hist} histograms; this search-wide adjustment gives scalar thresholds $\tau_{\text{split}}(L, N)$ at each test's actual N, while histograms reject at $p_{\text{val}} < \alpha/(2N_{\text{hist}})$.

An arm specifies shot allocation: the full-shot arm uses $N = 8{,}192$ per elementary test per role, whereas the fixed-budget arm divides the test-catalogue budget, $12 \times 8{,}192 = 98{,}304$ shots per role, across all J tests in fixed order. This design samples each elementary test separately, although a practical implementation could share bit

strings among statistics at the same circuit setting and obtain more shots per setting, an allocation that was not evaluated. Every searched test is sampled without stopping or reusing a control, and Bernoulli statistics use the exact two-bin pushforward of the full multinomial law. The archived experiment uses 1,000 repetitions, independent subject streams and recorded seeds from the permuted congruential generator PCG64 (Online Resource 1, Section S8.1).

Per-subject error control covers each subject's whole search without dividing α across the subject set, but the Pearson histogram test remains asymptotic and gives no exact guarantee of its level at a finite sample size, especially with sparse fixed-budget bins. The set-wide false-positive frequency is the fraction of repetitions detecting at least one equivalent-build subject in the stated set.

A mixed local measurement setting chooses Z, X or Y independently on each qubit, whereas homogeneous settings use the same measurement axis on every qubit. A Pauli string is a tensor product of local Pauli operators and identities, and its correlation weight counts the non-identity factors.

For phase-only sensitivity, single-qubit $|+\rangle$ inputs and the all-$|+\rangle$ input before each slot extend beyond the executor contract and can violate the fixed-zero masks.

Use of AI tools.

We used generative AI tools (Claude Opus 5.5, OpenAI Codex, ChatGPT and Gemini 4 Argon) as follows. For translation, English editing and packaging support, the authors manually reviewed all outputs.

4 Results for RQ1: Why do subjects survive the test catalogue?

We report the causes of survival under each contract and compare fixed-wrapper outcomes with native property tests.

4.1 Census and numerical checks

Table 3 and Fig. 2 report subject counts for both contracts of each archive, with behaviour-group counts in the table; QuCheck has 50 silent subjects and Muskit has 515. QuCheck's non-equivalent silent subjects are contract-limited under the program-semantic contract. This classification concerns only 3-qubit Deutsch–Jozsa, Grover and teleportation builds and 2-qubit superdense coding (Table 2). The separate native QuCheck property-test run detects one Grover subject (Section 4.3), while the other eight contract-limited subjects remain undetected in that run. All 40 subjects declared semantics-preserving by the upstream archive pass the full-circuit equivalence check, and one further subject also has equivalent builds.

Decisions from stored shot counts agree with exact probabilities in all 1,440 QuCheck and 13,632 Muskit subject–test cells, and neither mutant set has a threshold-limited or measurement-basis-limited subject. Every included build is unitary, and no build triggers the stop for conflicting diagnostics in Algorithm 1. Muskit has 20 positive subthreshold cells from five QRAM subjects, each detected in another basis,

Table 3 The census reports class counts by input contract in both archives; counts are subjects (behaviour groups)

		QuCheck		Muskit	
Class	Name	Executor	Program-semantic	Executor	Slot-consistent
E	Equivalent build	41 (4)	41 (4)	154 (3)	154 (3)
A	Admissible-input witness	2 (2)	0 (0)	130 (95)	8 (7)
C	Contract-limited	0 (0)	9 (9)	0 (0)	171 (116)
P	Phase-only	7 (7)	0 (0)	181 (97)	132 (69)
D	Detected	70 (64)	70 (64)	621 (382)	621 (382)
O	Summary-limited	0 (0)	0 (0)	50 (31)	50 (31)
M	Measurement-basis-limited	0 (0)	0 (0)	0 (0)	0 (0)
T	Threshold-limited	0 (0)	0 (0)	0 (0)	0 (0)
I	Input-limited total	9 (9)	9 (9)	311 (192)	311 (192)

I is the A + C + P aggregate (Table 1). QuCheck totals 120 subjects (77 groups); Muskit totals 1,136 (608). Numerical criteria: Section 3.3. Supporting counts: Online Resource 1, Tables S19 and S46.

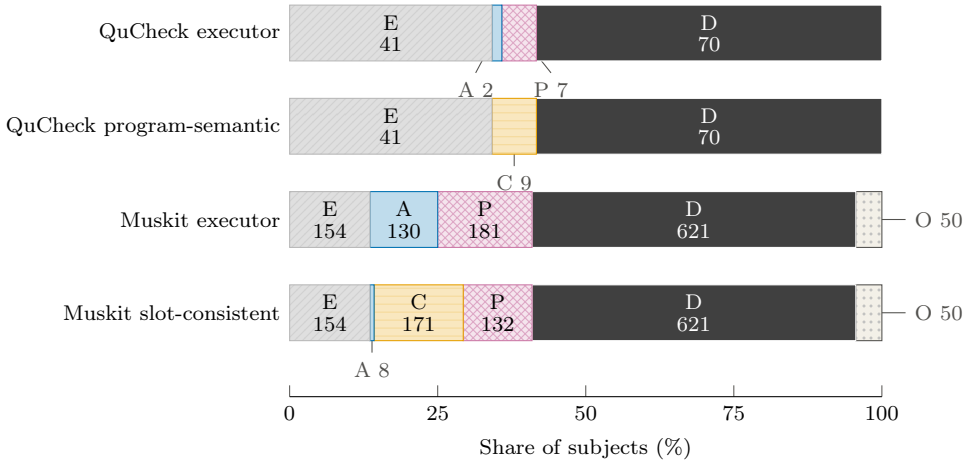


Fig. 2 Census shares and subject counts by input contract (class names in Table 1; zero-count classes omitted) show that Muskit's slot-consistent contract reduces admissible-input witnesses from 130 to 8 while existing detections stay fixed

but these cells add no threshold-limited subject because the class is assigned at the individual subject level (Online Resource 1, Table S1). Multiplying or dividing any numerical tolerance by 10 changes no class. Raising the detection threshold tenfold moves 45 QuCheck and 358 Muskit detected subjects to the threshold-limited class (Online Resource 1, Table S21). The census therefore separates why a subject is silent

from differences that stay below threshold in individual tests. For static builds of at most 7 qubits, the census used 22.21 s and peak resident memory 401,912 KiB; dense double-precision complex storage needs 16×4^n bytes per unitary.

4.2 What the contract changes

Muskit's source documents no fixed-zero workspace or excluded-input rule; the example input–output pairs in Muskit's documentation do not establish a qubit map or forbid other inputs. The slot-consistent contract restricts alternative inputs through the masks in Section 3.2, and coherent gates inside the program body do not change the application input domain defined before those operations. Source and slot audits, including excluded-slot sensitivities, appear in Online Resource 1, Table S46.

Under QuCheck's program-semantic contract, no silent subject has an admissible input change that detects it: the Grover distinguishing input $|4\rangle$, for example, changes a workspace qubit required to start at zero and belongs only to the executor contract. The inserted gate in the contract-limited teleportation subject acts on a workspace qubit already in $|+\rangle$; its build agrees for arbitrary admissible data input. Deutsch–Jozsa's slots cover four of the eight constant or balanced 2-bit oracles, but checking the missing four adds no detection: the silent subjects remain 10 equivalent-build and 3 contract-limited subjects (Online Resource 1, Section S4.1). The admissible input domain determines whether an input change can address non-detection.

4.3 Native QuCheck property-test comparison

A separate central processing unit (CPU) execution preserves QuCheck's native generators, preconditions and assertions, although an initial execution error required a corrected run with native defaults. The corrected run kept properties, inputs, seeds and statistics unchanged under a separately hashed protocol. Execution errors supply no positive or negative detection outcome. The corrected run detects 68 of 120 subjects and agrees with the corresponding fixed-wrapper outcome on 116 of 120, with native-only detection of one Grover subject and catalogue-only detections of another Grover subject and two teleportation subjects. Native detections include 0 of 41 equivalent-build subjects and 0 of 4 release controls. Both attempts, protocol hashes and counts remain in Online Resource 1, Section S7.2 and Table S45. Their stored count records lack the assertion-to-circuit map needed to derive native p-values from those records alone. Unmatched budgets and generated circuit ranges prevent a matched comparison of test effectiveness (Section 7).

Answer to RQ1

Under its program-semantic contract, QuCheck's 50 silent subjects comprise 41 equivalent-build and 9 contract-limited subjects. Muskit's 515 silent subjects comprise 154 equivalent-build, 50 summary-limited and 311 input-limited subjects; the slot-consistent contract reduces admissible-input witnesses from 130 to 8.

5 Results for RQ2: Which changes to tests detect silent subjects?

We compare admissible-input witnesses, fresh-shot detections, shot costs and the information lost by each test family.

5.1 Exact-law detections, fresh-shot detections and search cost

Table 4 separates deterministic exact-law detections from fresh-shot detections after adjustment for each subject's entire search. Each contract has its own admissible-input denominator: executor 130 subjects (95 groups), slot-consistent 8 (7). The fresh-shot columns count subjects whose estimated frequency of detection with new shots is at least 0.95, with the same counts at a detection-frequency cut-off of 0.5, whereas exact-law detections use numerical criteria rather than sampled power. Table 5 reports set-wide false-positive frequencies over 195 equivalent-build subjects in 7 groups, pooling 41 QuCheck and 154 Muskit subjects. Contract labels in Table 5 apply only to Muskit, because QuCheck retains its executor input domain in both comparisons. Each group is represented by its member with the smallest source identifier, and a group frequency tests only that member. The two contract comparisons use separate seeds, so QuCheck's frequencies differ even though its input domain stays fixed.

Table 4 Cumulative rungs report exact-law and fresh-shot detections for Muskit's admissible-input witnesses

			Fresh-shot	
Contract	Rung	Exact-law detections	Full-shot arm	Fixed-budget arm
Executor	input	68 (58)	68 (58)	0 (0)
Executor	+ pairs	76 (65)	76 (65)	0 (0)
Executor	+ histogram	118 (89)	118 (89)	0 (0)
Slot-consistent	input	8 (7)	8 (7)	6 (5)
Slot-consistent	+ pairs	8 (7)	8 (7)	8 (7)
Slot-consistent	+ histogram	8 (7)	8 (7)	8 (7)

Counts are subjects (behaviour groups); exact-law and full-shot counts coincide in every row. Denominators: executor 130 (95); slot-consistent 8 (7). Fresh-shot detection frequency is at least 0.95 across 1,000 repetitions. Full scopes and intervals: Online Resource 1, Tables S51–S54.

The full-shot arm uses $J \times 8{,}192$ shots per role versus $12 \times 8{,}192$ in the test catalogue, giving a cost multiplier of $J/12$, where $J = L + N_{\text{hist}}$. Executor CE and LittleBV have $J = 17{,}664$ and cost 1,472 test-catalogue budgets per role, and QRAM costs 3,840; slot-consistent searches cost 184 budgets for CE and LittleBV and 240 for QRAM. For Muskit's fixed-budget arm under the executor contract, $2 \leq N \leq N_{\max} = 6$ and $K \geq K_{\min} = 16{,}896$. The split-alpha threshold defined in Section 3.4

Table 5 Set-wide false-positive frequencies vary by arm and rung over equivalent-build subjects from both archives

		Full-shot arm		Fixed-budget arm	
Muskit contract	Rung	Subjects	Groups	Subjects	Groups
Executor	input	0.000	0.000	0.000	0.000
Executor	+ pairs	0.000	0.000	0.000	0.000
Executor	+ histogram	0.071	0.005	0.013	0.002
Slot-consistent	input	0.000	0.000	0.000	0.000
Slot-consistent	+ pairs	0.000	0.000	0.000	0.000
Slot-consistent	+ histogram	0.069	0.002	0.016	0.001

Frequencies count repetitions detecting at least one equivalent-build subject among 195 subjects (7 groups), pooled across QuCheck and Muskit. QuCheck retains its executor domain in both comparisons; contract rows name the Muskit domain. Groups use their member with the smallest source identifier. All frequencies use 1,000 repetitions. Intervals: Online Resource 1, Tables S51–S54.

therefore satisfies

$$\tau_{\text{split}}(K, N) \geq \tau_{\text{split}}(K_{\min}, N_{\max}) > 1. \tag{3}$$

No probability gap can exceed 1, so scalar tests cannot reject under the bound in Eq. (3). In Eq. (2), each summand is at most $a_z + c_z$, giving $Q \leq 2N$. Thus CE and LittleBV have $Q \leq 12$ and $\Pr(\chi^2_1 \geq 12) > \alpha/(2N_{\text{hist}})$, while QRAM has $Q \leq 6$ and $\Pr(\chi^2_1 \geq 6) > \alpha/(2N_{\text{hist}})$. These bounds hold for any positive degrees of freedom, with $p_{\text{val}} = 1$ otherwise, so neither scalar nor histogram tests can reject at the test-catalogue budget. The archived fixed-budget simulation checks this consequence of the bounds, while the slot-consistent fixed-budget results remain empirical under their larger per-test allocations (Online Resource 1, Section S8.1).

The input and + pairs rungs have zero false-positive events, so all reported events come from the + histogram rung. For Muskit alone, the set-wide false-positive frequencies at the + histogram rung are 0.052 for the full-shot arm and 0.000 for the fixed-budget arm under the executor contract, versus 0.059 and 0.002 under the slot-consistent contract. The pooled set-wide false-positive frequency of the fixed-budget arm under the executor contract comes entirely from QuCheck, whose smaller searches receive more shots per test. All subgroup frequencies and intervals appear in Online Resource 1, Tables S51–S54, conditional on the archived noiseless laws.

5.2 Information discarded by existing statistics

At catalogue inputs, the split-alpha Pearson comparison detects all 50 summary-limited subjects in 31 groups, all CE, without pairs. It uses fixed output-bit-string bins and no input-specific bin or pair selection, loses no existing detection and detects 0 of 195 equivalent-build subjects in stored shots. The pair (q_2, q_3) explains all these detected correlations in the supplied CE subjects, while every other CE pair stays below the original threshold. The source edits comprise 48 additions and 2 replacements at 6 gate-call boundaries (Online Resource 1, Table S20). A differing histogram

defines the summary-limited class; the Pearson detections and the shared (q_2, q_3) pair are findings for these CE subjects, not a general success rate for test changes. The independent-draw calibration of the split-alpha Pearson comparison at catalogue inputs and the fresh-shot search use different test families, whose frequencies and intervals appear separately in Online Resource 1, Tables S44 and S51. Full histograms detect differences that the existing marginal and parity statistics discard in these CE subjects.

5.3 Measurement bases and phase-only inputs

The 12 QRAM admissible-input witnesses remaining after the + histogram rung under the executor contract change output states under other inputs while retaining identical full histograms in each homogeneous setting (Section 3.4). They are therefore measurement-basis-limited at those inputs, although no subject belongs to that class at catalogue inputs (Table 3). For every residual subject, all $3n + 3^2\binom{n}{2} = 210$ non-identity Pauli statistics of weight at most two agree over all 128 basis inputs and 4 slots, for $n = 7$. For each residual subject, an exhaustive 16,384-coefficient Pauli expansion at one admissible distinguishing input held fixed finds a weight-four Pauli correlation, without stopping at the first gap. This minimum is conditional on that fixed input, not a global minimum across inputs; the negative result for weight at most two across all inputs is a separate check (Online Resource 1, Table S42).

The phase-only sensitivity detects 181 of 181 executor phase-only subjects with existing statistics, which separates phase differences that coherent inputs can reveal from missing local measurement correlations (Section 3.4).

Answer to RQ2

Full-shot input search with pairwise-parity and full-histogram tests detects 118/130 executor witnesses at 1,472–3,840 test-catalogue budgets per role. Fixed-budget search detects none; both detect all 8 slot-consistent witnesses. Pearson histogram tests detect all 50 summary-limited subjects; 12 undetected executor witnesses require mixed local measurement settings.

6 Discussion and implications

We discuss how the census guides testing decisions and relates to earlier quantum mutation and equivalence studies. The census orders a tester's decisions: check full-circuit equivalence, establish the admissible input domain, then distinguish a missing input from lost histogram information or a missing measurement setting. Muskit's narrow slot preparation and its oracle, which keeps only marginals and parity, create the conditions for admissible-input witnesses, phase-only subjects and summary-limited subjects, but their shares do not characterise Muskit's native testing or tools that already compare full histograms.

Earlier mutation studies examine equivalent mutants and oracle choice (Fortunato et al., 2022a), and relate survival to edit and program characteristics (Mendiluze Usandizaga et al., 2025), whereas the census separates full-circuit equivalence, input

restrictions and discarded histogram information to guide testing decisions for the supplied builds. Transpiler-based equivalence checking (Campos & Miranskyy, 2026) can address the first cause but does not determine which non-equivalent differences fall within an input contract. For quantum software quality assurance, QuCheck and Muskit tool builders should make admissible preparations and retained histogram information explicit, so testers can choose an input or oracle change for the relevant cause.

At larger scales, a circuit-equivalence checker (Burgholzer & Wille, 2021) can replace the dense-unitary first step, while selected admissible inputs can show that a subject is an admissible-input witness but cannot establish that no distinguishing input exists. Such approximations would change the exhaustive scope and need separate validation.

7 Threats to validity

We assess construct, internal, external and conclusion validity using the taxonomy of Wohlin et al. (2012).

7.1 Construct validity

A separate NumPy code path reads circuit text and computes gates, rotations and statistics, while sharing supplied builds and control identities. These shared inputs limit what agreement shows: an incorrect circuit or control can survive both the original and the NumPy code paths. The NumPy path gives distinct parsing and arithmetic under the same study protocol, rather than validation by another team. Marginals, parity and homogeneous measurement bases do not determine the full output state.

7.2 Internal validity

All census, test-change and sensitivity analyses are post hoc and exploratory after exposure, meaning earlier outcomes were seen before analysis design. Enumerating inputs and internally hashing protocols document ordering, but do not establish independent validation on programs unseen during design. We fixed descriptive checks of test-suite gains on Muskit after seeing QuCheck outcomes but before seeing Muskit outcomes; those checks support neither RQ (Online Resource 1, Section S3). The pre-outcome exclusion of unbuildable subjects described in Section 3.2 leaves analysis over all 1,172 planned subjects incomplete because outcomes for excluded subjects were not analysed.

We reduced stored counts and circuit outputs without rerunning the original QuCheck simulator, whose interpreter patch, binary, driver and firmware remain unpinned. Muskit has a pinned environment and preserved execution records; numerical analyses recompute preserved builds (Online Resource 1, Table S29). The inventory of original-binary dependencies distinguishes stored-count reduction and CPU census reconstruction from fresh original-runtime execution (Online Resource 1, Table S55).

Stored-count reduction and CPU reconstruction do not require the original simulator binary, whereas fresh original-runtime execution remains sensitive to its unpinned environment. Incorrect release controls or circuit-text adapters, which reconstruct circuits from their text, could corrupt the census, so we check them. All 36 of 36 analytic release-control comparisons pass across nine programs and four slots, including both QuCheck development programs; 4,556 of 4,556 circuit-text adapter comparisons also pass. The latter check the arithmetic reconstruction against the supplied builds (Online Resource 1, Table S23), but CE and QRAM expectations come from their source gates rather than independent application specifications.

7.3 External validity

Shared Bell code contributes twice as many subjects as each of the other two evaluation lineages, while Muskit's three programs share a repository and mutation system. Source edits, identical OpenQASM build texts and behaviour groups do not establish independent faults. Neither mutant set is a validated real-fault sample, and program and hardware differences cannot isolate a hardware effect or predict noisy-device behaviour. The census does not support intermediate measurement, reset, classical control or hardware noise. Native property-based testing changes generated circuit ranges and total budget, so its outcome comparison does not evaluate matched test effectiveness. Native non-detection does not prove a subject's class across the native generators' full circuit ranges. The procedure has not been evaluated on other programs or against Quito or MutTG.

7.4 Conclusion validity

Full-circuit equivalence checks use floating-point tolerances and allow global phase without giving symbolic proofs, while the fresh-shot scalar bound covers each subject's entire stated search rather than all subjects at once. Pearson histogram tests use asymptotic tails, whose approximation is least reliable with sparse fixed-budget bins; at catalogue inputs, 899 of 15,072 stored-shot cells contain bins with expected counts below 5 (Online Resource 1, Section S7.2). The 1,000 repetitions leave Monte Carlo uncertainty in the reported frequencies; the reported intervals quantify that uncertainty. Stored-shot non-detection of equivalent-build subjects does not establish specificity because release controls were reused, whereas the simulations use separate control draws. Set-wide false-positive events limit simultaneous specificity claims but do not change admissible-input-witness counts calculated separately.

8 Conclusion

We conclude that silent subjects in these supplied builds fall into equivalent-build, input-limited and summary-limited classes that require different testing decisions. Under QuCheck's program-semantic contract, 41 silent subjects have equivalent builds and 9 are contract-limited. Muskit's admissible-input witnesses fall from 130 under the executor contract to 8 under the slot-consistent contract, without changing existing detections. The full-shot arm across the input, + pairs and + histogram rungs

detects 118 of 130 executor admissible-input witnesses, and Pearson histogram tests detect all 50 summary-limited subjects at catalogue inputs without pair selection. The residual QRAM subjects need mixed local measurement settings at the examined inputs. Muskit's fixed-budget arm under the executor contract cannot reject at the test-catalogue budget by construction. A tester should establish admissible inputs and inspect full histograms before interpreting survival as equivalence, although these exploratory findings remain limited to the supplied builds and contracts.

Future work should assess the procedure on other programs and noisy devices, validate scalable equivalence checks and evaluate allocations that share measured bit strings across statistics.

Statements and Declarations

The following statements cover ethics, availability, funding, competing interests, authorship and AI use.

Ethics and consent

Not applicable. This study involves no human participants or animals.

Data, materials and code availability

Online Resource 1 contains supporting methods, tables and integrity records, indexed in its opening resource map. Online Resource 2 contains the replication package with stored counts, circuit builds, numerical analyses, provenance and instructions for reproducing the reported numbers. The material separates stored-count reduction, numerical recomputation of preserved builds and fresh simulator execution. Permanent deposition is deferred until acceptance. The study package and LaTeX sources are also available at https://github.com/insui12/qsa-replication. Reuse restrictions and third-party licences accompany the package.

Funding

No funding was received for conducting this study.

Competing interests

The authors have no competing interests to declare that are relevant to the content of this article.

Authors' contribution statements

Seojin Choi designed the study, conducted the experiments and analyses, and wrote the manuscript. Geunseok Yang supervised the work and provided critical feedback on the manuscript. All authors read and approved the final manuscript.

Use of AI tools

See the Use of AI tools paragraph in Section 3.

References

Ali, S., Arcaini, P., Wang, X., & Yue, T. (2021). Assessing the effectiveness of input and output coverage criteria for testing quantum programs. In *2021 14th IEEE Conference on Software Testing, Verification and Validation (ICST)* (pp. 13–23). IEEE. https://doi.org/10.1109/ICST49551.2021.00014

Burgholzer, L., & Wille, R. (2021). Advanced equivalence checking for quantum circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, *40*(9), 1810–1824. https://doi.org/10.1109/TCAD.2020.3032630

Campos, J., & Miranskyy, A. (2026). *Quantum mutant equivalence via transpilation*. arXiv. https://doi.org/10.48550/arXiv.2606.26604

Fortunato, D., Campos, J., & Abreu, R. (2022a). Mutation testing of quantum programs: A case study with Qiskit. *IEEE Transactions on Quantum Engineering*, *3*, Article 2500517. https://doi.org/10.1109/TQE.2022.3195061

Fortunato, D., Campos, J., & Abreu, R. (2022b). QMutPy: A mutation testing tool for quantum algorithms and applications in Qiskit. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)* (pp. 797–800). ACM. https://doi.org/10.1145/3533767.3543296

Honarvar, S., Mousavi, M. R., & Nagarajan, R. (2020). Property-based testing of quantum programs in Q#. In *Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops (ICSEW)* (pp. 430–435). ACM. https://doi.org/10.1145/3387940.3391459

Huang, Y., & Martonosi, M. (2019). Statistical assertions for validating patterns and finding bugs in quantum programs. In *Proceedings of the 46th International Symposium on Computer Architecture (ISCA)* (pp. 541–553). ACM. https://doi.org/10.1145/3307650.3322213

Jia, Y., & Harman, M. (2011). An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, *37*(5), 649–678. https://doi.org/10.1109/TSE.2010.62

Li, Y., Shao, M., Zhao, J., & Wang, Q. (2026). A methodological analysis of empirical studies in quantum software testing. *ACM Transactions on Software Engineering and Methodology*. Advance online publication. https://doi.org/10.1145/3819590

Mendiluze, E., Ali, S., Arcaini, P., & Yue, T. (2021). Muskit: A mutation analysis tool for quantum software testing. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)* (pp. 1266–1270). IEEE. https://doi.org/10.1109/ASE51524.2021.9678563

Mendiluze Usandizaga, E., Ali, S., Yue, T., & Arcaini, P. (2025). Quantum circuit mutants: Empirical analysis and recommendations. *Empirical Software Engineering*, *30*(4), Article 100. https://doi.org/10.1007/s10664-025-10643-z

Paltenghi, M., & Pradel, M. (2023). MorphQ: Metamorphic testing of the Qiskit quantum computing platform. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)* (pp. 2413–2424). IEEE. https://doi.org/10.1109/ICSE48619.2023.00202

Paltenghi, M., & Pradel, M. (2024). *A survey on testing and analysis of quantum software*. arXiv. https://doi.org/10.48550/arXiv.2410.00650

Papadakis, M., Kintis, M., Zhang, J., Jia, Y., Le Traon, Y., & Harman, M. (2019). Mutation testing advances: An analysis and survey. In A. M. Memon (Ed.), *Advances in Computers* (Vol. 112, pp. 275–378). Elsevier. https://doi.org/10.1016/bs.adcom.2018.03.015

Pontolillo, G., Mousavi, M. R., & Grzesiuk, M. (2025). *QuCheck: A property-based testing framework for quantum programs in Qiskit*. arXiv. https://doi.org/10.48550/arXiv.2503.22641

Pontolillo, G., Mousavi, M. R., & Grzesiuk, M. (2026). QuCheck: A property-based testing framework for quantum programs in Qiskit. *ACM Transactions on Quantum Computing*, *7*(4), Article 24. https://doi.org/10.1145/3815169

Saad, A. M., Sabit, A. A., Brandenburgh, J., Wang, J., Zhang, Q., & Hossain, S. B. (2026). *Are we testing the wrong quantum bugs? A study of fault representation gaps in quantum software testing*. arXiv. https://doi.org/10.48550/arXiv.2610.22089

Wang, X., Yu, T., Arcaini, P., Yue, T., & Ali, S. (2022). Mutation-based test generation for quantum programs with multi-objective search. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)* (pp. 1345–1353). ACM. https://doi.org/10.1145/3512290.3528869

Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., & Wesslén, A. (2012). *Experimentation in software engineering*. Springer. https://doi.org/10.1007/978-3-642-29044-2

Yoo, S., & Harman, M. (2012). Regression testing minimization, selection and prioritization: A survey. *Software Testing, Verification and Reliability*, *22*(2), 67–120. https://doi.org/10.1002/stvr.430